实 验 报 告

实验名称： 实验七 CNN 做手写体识别

课程名称： 认知科学与类脑计算

实验地点： K2 227

实验日期： 2025/9/12

班 级： 23 级智能班

姓 名： 宋浩宇

学 号： 202300130183

一 实验目的：

加深对卷积神经网络模型的理解，能够使用卷积神经网络模型解决简单问题

二 实验环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux
编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11
Python 解释器：Python 3.9
Pytorch 版本：2.8.0
CUDA 版本：12.8.97

硬件环境：
CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz
内存：32.0 GB (31.6 GB 可用)
磁盘驱动器：NVMe WD_BLACKSN850X2000GB
显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU
CUDA 核心数：7424

三 实验内容：

根据卷积神经网络模型的相关知识，使用 Python 语言实现一个简单卷积神经网络模型，该模型能够实现手写数字识别。

四 实验步骤：

1. 实验程序：

```
import numpy as np
from tqdm import tqdm
from torchvision import datasets, transforms
import torch

MNIST_PATH = r'F:\DATASET\MNIST\mnist_dataset_dir'
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
batch_size = 64
train_data = datasets.MNIST(root=MNIST_PATH,
                            train=True,
                            transform=transform,
                            download=True)
test_data = datasets.MNIST(root=MNIST_PATH,
                           train=False,
                           transform=transform,
                           download=True)
```

```
train_gen = torch.utils.data.DataLoader(train_data,
                                        batch_size=batch_size,
                                        shuffle=True)
test_gen = torch.utils.data.DataLoader(test_data,
                                       batch_size=batch_size,
                                       shuffle=False)
class LeNet(torch.nn.Module):
    def __init__(self, num_classes=10):
        super(LeNet, self).__init__()
        self.conv1 = torch.nn.Conv2d(kernel_size=5,
padding=2,in_channels=1, out_channels=6)
        self.s2 = torch.nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv3 = torch.nn.Conv2d(kernel_size=5, padding=0,
in_channels=6, out_channels=16)
        self.s4 = torch.nn.AvgPool2d(kernel_size=2, stride=2)
        self.fc1 = torch.nn.Linear(in_features=16 * 5 * 5,
out_features=120)
        self.fc2 = torch.nn.Linear(in_features=120, out_features=84)
        self.fc3 = torch.nn.Linear(in_features=84,
out_features=num_classes)
        self.get_layer_info(torch.randn(1, 28, 28))
    def get_layer_info(self, x):
        x = self.conv1(x)
        print(f"Conv1 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU1 output shape: {x.shape}")
        x = self.s2(x)
        print(f"AvgPool1 output shape: {x.shape}")
        x = self.conv3(x)
        print(f"Conv2 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU2 output shape: {x.shape}")
        x = self.s4(x)
        print(f"AvgPool2 output shape: {x.shape}")
        x = torch.flatten(x)
        print(f"Flatten output shape: {x.shape}")
        x = self.fc1(x)
        print(f"FC1 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU3 output shape: {x.shape}")
        x = self.fc2(x)
        print(f"FC2 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU4 output shape: {x.shape}")
```

```
        x = self.fc3(x)
        print(f"FC3 output shape: {x.shape}")
        return x
    def forward(self, x):
        x = self.conv1(x)
        x = torch.nn.functional.relu(x)
        x = self.s2(x)
        x = self.conv3(x)
        x = torch.nn.functional.relu(x)
        x = self.s4(x)
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = torch.nn.functional.relu(x)
        x = self.fc2(x)
        x = torch.nn.functional.relu(x)
        x = self.fc3(x)
        return x
model = LeNet()
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print(f"训练设备: {device}")
for epoch in range(10):
    running_loss = 0.0
    for i, data in enumerate(tqdm(train_gen), 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f"Epoch {epoch+1} loss: {running_loss/len(train_gen)}")
correct = 0
total = 0
model.eval()
for i, data in enumerate(tqdm(test_gen), 0):
    inputs, labels = data
    inputs, labels = inputs.to(device), labels.to(device)
    outputs = model(inputs)
    _, predicted = torch.max(outputs.data, 1)
    total += labels.size(0)
```

```
    correct += (predicted == labels).sum().item()
print(f"Accuracy of the network on the 10000 test images: {100 * correct
/ total}%")
```

2. 程序实验结果：

```
  0%|          | 0/157 [00:00<?, ?it/s]Epoch 10 loss: 0.018509948178929692
100%|██████████| 157/157 [00:01<00:00, 147.49it/s]
Accuracy of the network on the 10000 test images: 99.12%
```

3. 分析和改进：

从结果上来看，模型的准确率非常高，模型已经非常优秀地完成了这个问题。改进的话，就是使用 numba 等方式来对训练速度做一个优化了。

五 实验小结：

模型的架构不是最简单的输入-卷积-池化-全连接-输出的架构，而是使用了更贴合人脑神经系统结构的 LeNet 架构（本质其实就是改进版的 CNN），训练出的模型的性能非常不错。